

Java EE – Fondamentaux du développement web



Ce que vous allez apprendre

01

Introduction à Java EE

Architecture web, environnement Eclipse et Tomcat.

02

Les Servlets

Cycle de vie, requêtes HTTP, sessions et filtres.

03

Les JSP

Pages dynamiques, EL, JSTL et architecture MVC.

04

L'API JDBC

Connexion BDD, requêtes SQL et pools de connexions.

05

Notions complémentaires

HTTP/2, bonnes pratiques et récapitulatif.

Public cible & Prérequis

À qui s'adresse cette formation ?

- Développeurs Java souhaitant créer des applications web
- Étudiants en informatique
- Développeurs intermédiaires consolidant leurs bases Java EE

Prérequis techniques

- Bases du langage Java (classes, objets, héritage)
- Notions de HTML et CSS
- Compréhension du modèle client-serveur
- JDK installé sur votre machine

Environnement de développement

Eclipse IDE

Téléchargez *Eclipse IDE for Enterprise Java Developers* pour le support natif Java EE.

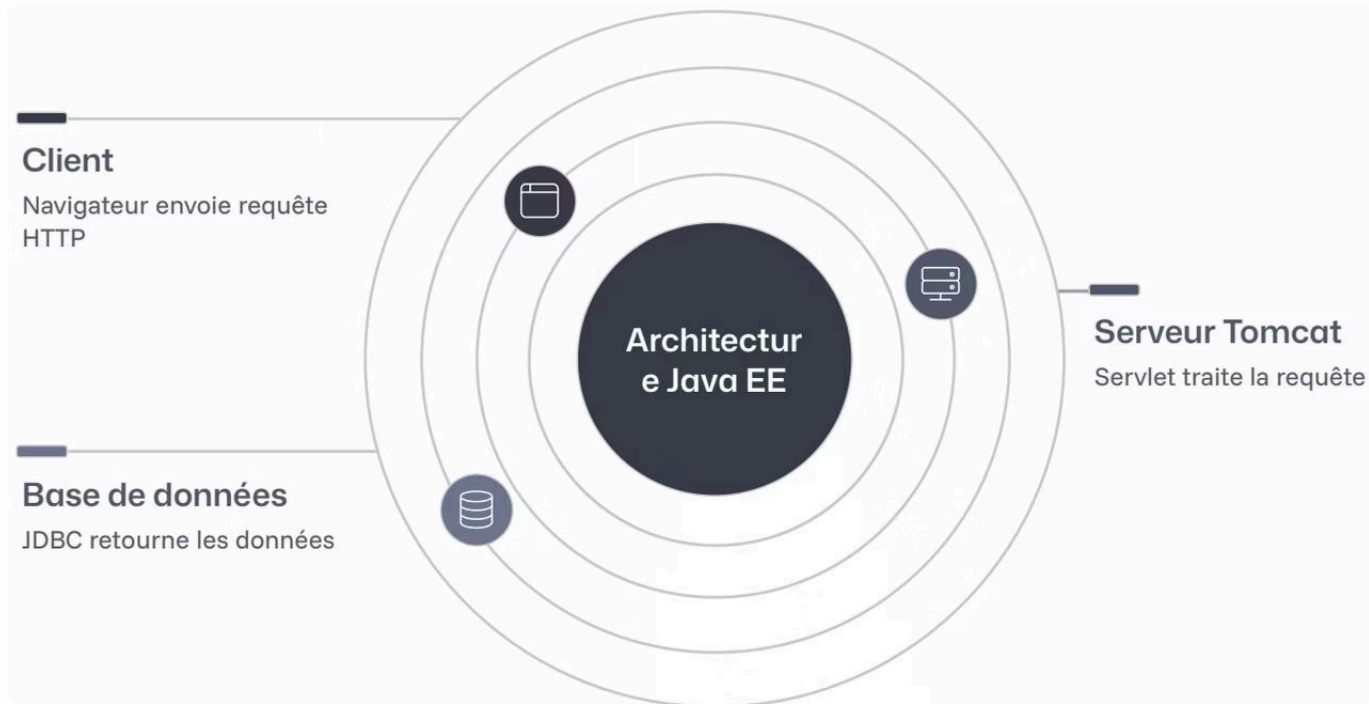
Apache Tomcat 9.x

Conteneur de servlets léger, compatible avec la spécification **Servlet 4.0** de Java EE 8.

JDK 8+

Indispensable. Utilisez au minimum la version 8 pour la compatibilité avec Java EE 8.

Architecture d'une application Java EE



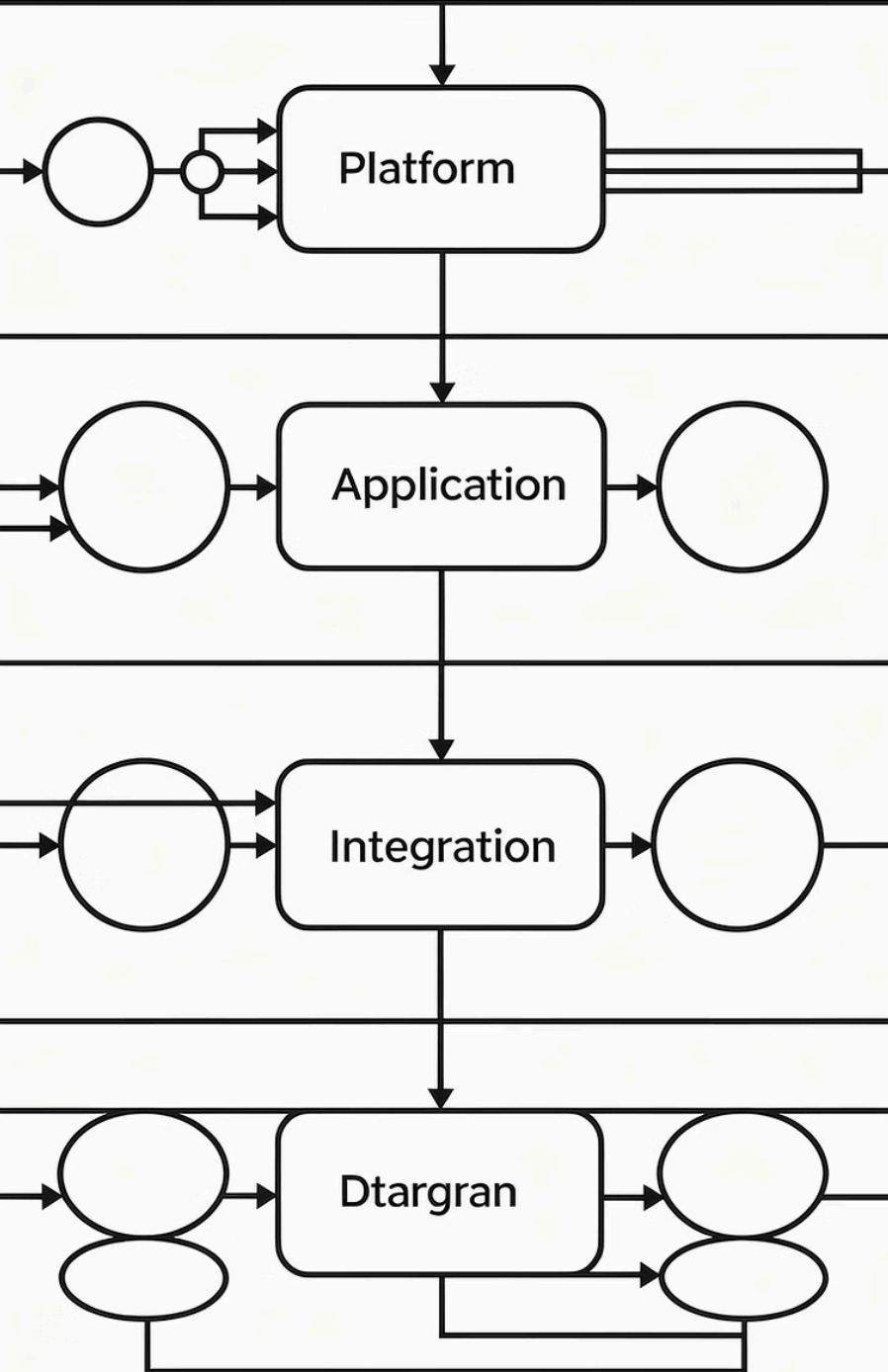
Modèle trois couches

Le navigateur émet des **requêtes HTTP**, Tomcat délègue le traitement aux Servlets ou JSP, qui interrogent la base de données via **JDBC** pour retourner une réponse dynamique.

- i** Ce modèle garantit la maintenabilité et la scalabilité de vos applications.

Enterprises

Java
Endpfation



Qu'est-ce que Java EE ?

Java EE (aujourd'hui **Jakarta EE**) étend Java SE avec des APIs dédiées aux applications d'entreprise : gestion des transactions, sécurité, accès aux données, communication web.

Servlet 4.0

Traitement des requêtes HTTP

JSP 2.3

Pages web dynamiques

JPA 2.2

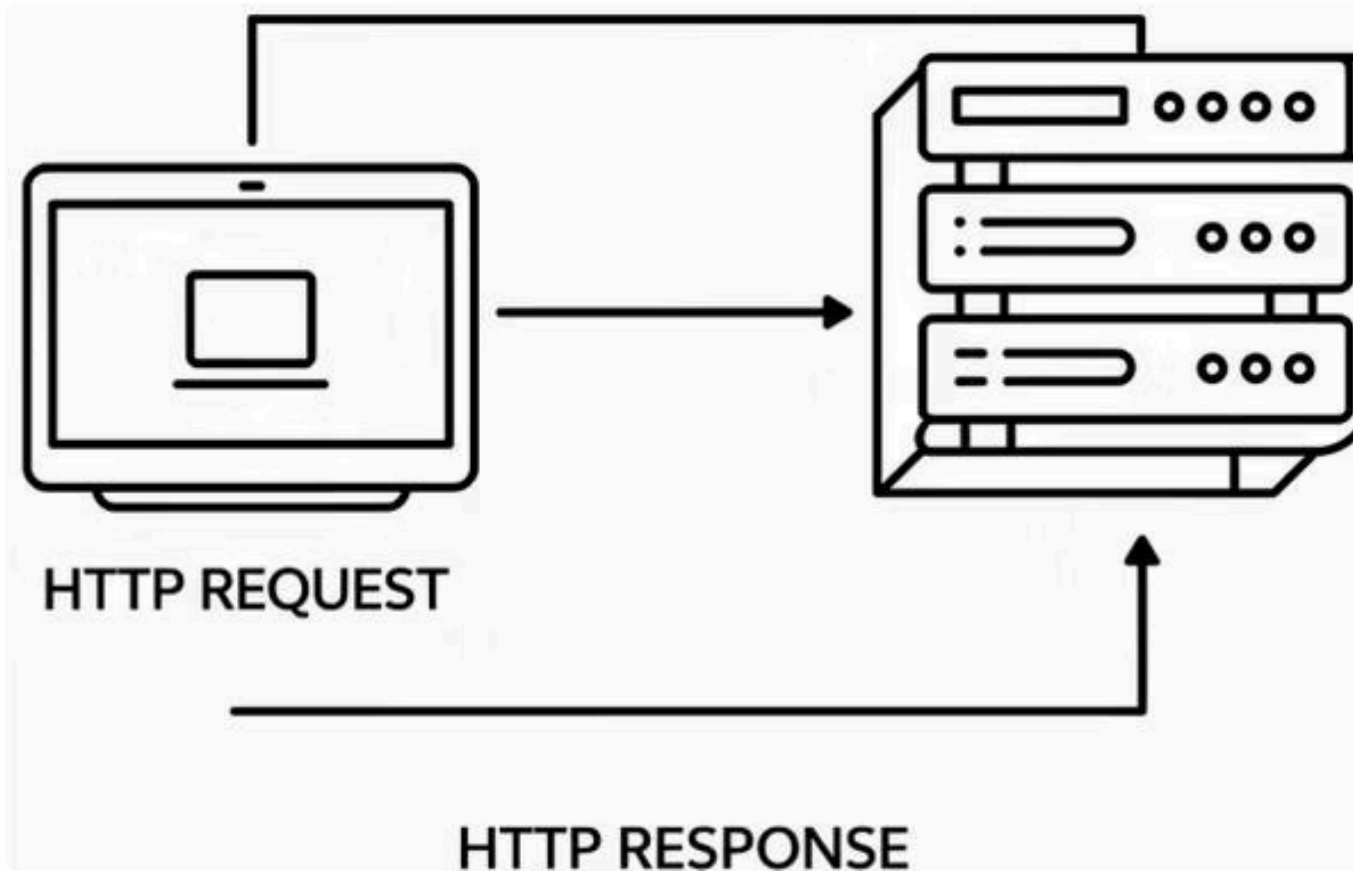
Persistence des données

JAX-RS 2.1

Services REST

📄 Cette formation se concentre sur les briques fondamentales : **Servlets, JSP et JDBC** — socle de tous les frameworks modernes comme Spring MVC.

Les Servlets



Une **Servlet** est une classe Java côté serveur qui répond aux requêtes HTTP. Elle est gérée par un **conteneur** (Tomcat) qui prend en charge instanciation, initialisation et destruction.

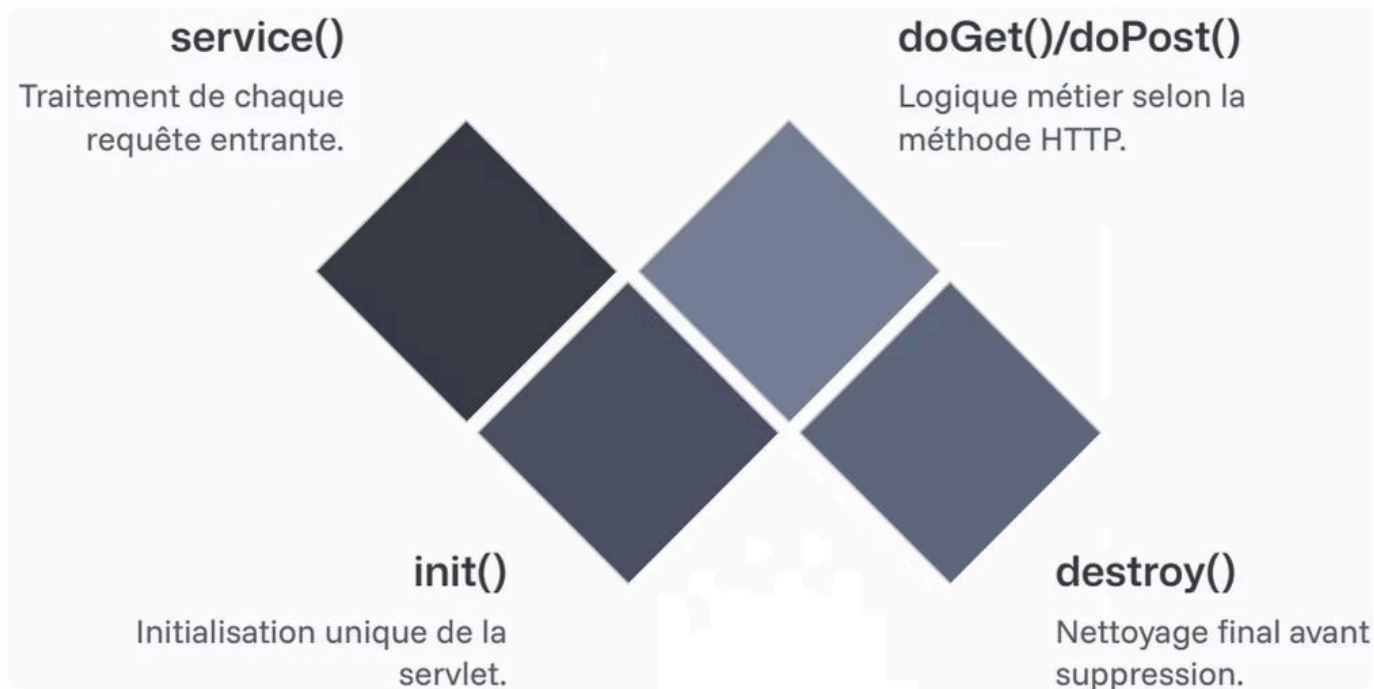
HttpServlet

Toute Servlet hérite de `HttpServlet` et surcharge `doGet()` / `doPost()`.

@WebServlet

Remplace la configuration XML depuis Servlet 3.0.

Cycle de vie d'une Servlet



1 init()

Appelée **une seule fois** au premier chargement.

2 service()

Invoquée à chaque requête, délègue à `doGet()` ou `doPost()`.

3 destroy()

Libère les ressources à l'arrêt du serveur.

⚠ Une seule instance traite toutes les requêtes simultanées. N'utilisez jamais de variables d'instance pour stocker des données de requête !

Votre première Servlet

```
@WebServlet("/bonjour")
public class BonjourServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws IOException {

        response.setContentType("text/html;charset=UTF-8");
        String nom = request.getParameter("nom");
        if (nom == null) nom = "visiteur";

        PrintWriter out = response.getWriter();
        out.println("
```

Bonjour, " + nom + " !

```
"; } }
```

 Accédez via : `http://localhost:8080/monapp/bonjour?nom=Alice`. Tomcat mappe automatiquement l'URL grâce à `@WebServlet`.

HTTP, Paramètres & Sessions

Verbes HTTP

Lisez les paramètres avec `request.getParameter("nom")` ou `getParameterValues()` pour les valeurs multiples.

GET

Lecture — paramètres visibles dans l'URL.

Sessions HTTP

HTTP est *stateless*. Pour maintenir un contexte utilisateur (panier, authentification), Java EE propose les **sessions HTTP**.

- `request.getSession()` — crée ou récupère la session
- `session.setAttribute("cle", obj)` — stocke un objet
- `session.setMaxInactiveInterval(1800)` — expiration
- `session.invalidate()` — déconnexion

POST

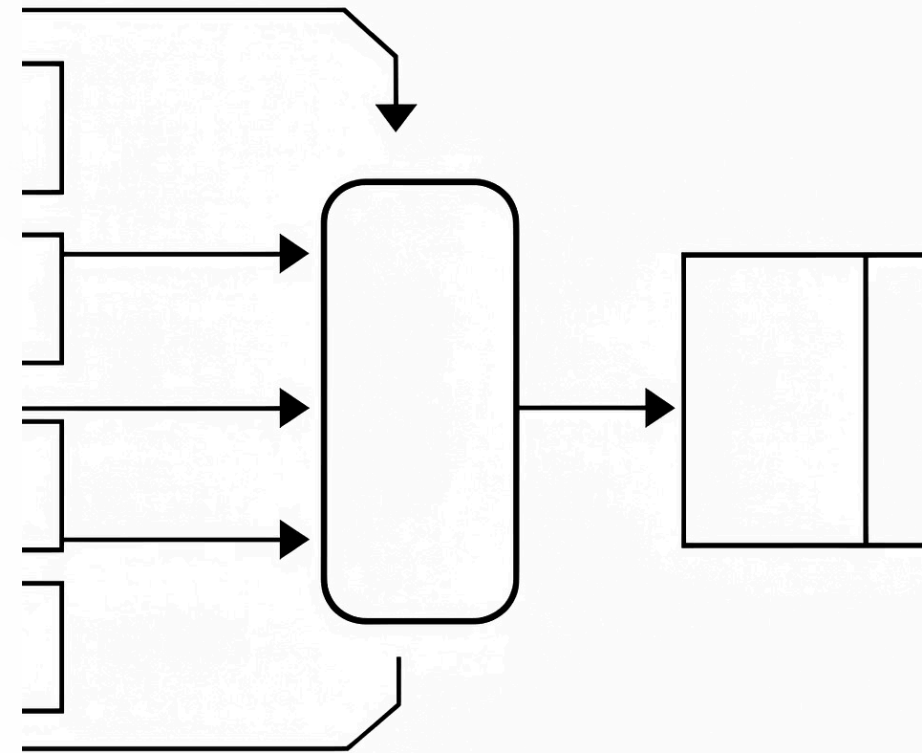
Envoi de données sensibles — paramètres dans le corps.

Les Filtres de Servlets

Un **filtre** intercepte les requêtes *avant* la Servlet et les réponses *après*. Idéal pour factoriser des traitements transversaux : authentification, journalisation, encodage, mise en cache.

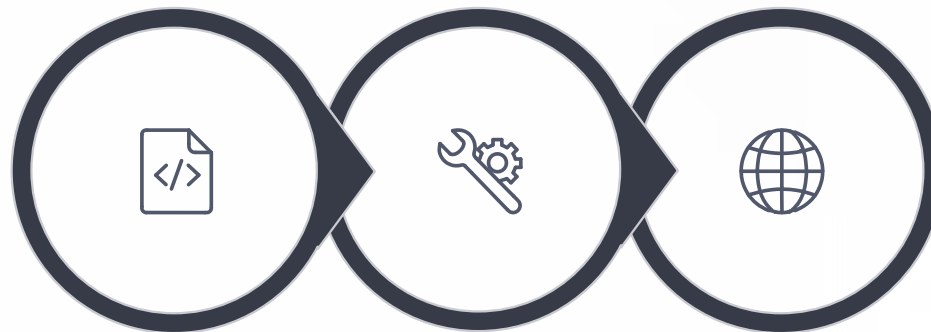
```
@WebFilter("/*")
public class EncodageFilter implements Filter {
    public void doFilter(ServletRequest req, ServletResponse res,
        FilterChain chain) throws IOException, ServletException
    {
        req.setCharacterEncoding("UTF-8");
        res.setCharacterEncoding("UTF-8");
        chain.doFilter(req, res); // Passe au filtre suivant ou à la Servlet
    }
}
```

i `@WebFilter("/*")` applique le filtre à toutes les URLs. Restreignez avec `@WebFilter("/admin/*")`.



Les JSP – Pages dynamiques

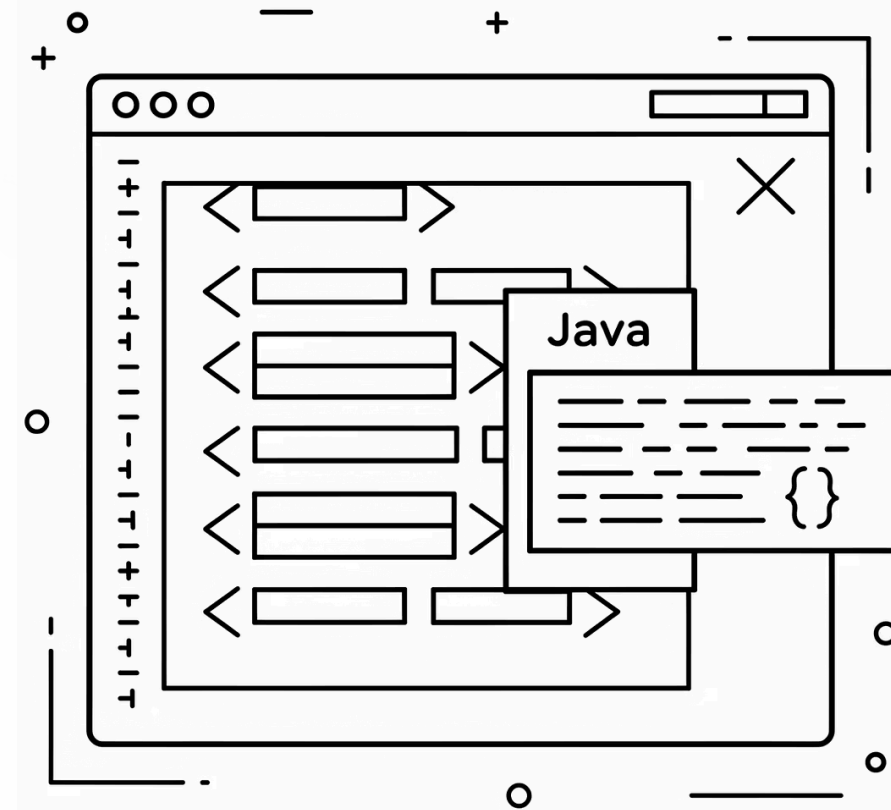
Les **JavaServer Pages (JSP)** combinent HTML statique et éléments Java dynamiques. Tomcat compile automatiquement la JSP en Servlet au premier accès.



Fichier JSP

Compilation

Exécution



Syntaxe des JSP

1

Scriptlets `<% ... %>`

Blocs de code Java. À éviter dans les JSP modernes — préférez JSTL et EL.

2

Expressions `<%= ... %>`

Évaluent une expression Java et insèrent son résultat dans le HTML.

3

Directives `<%@ ... %>`

Configurent la JSP : encodage, imports, bibliothèques de balises.

4

Expression Language `${...}`

Accède aux attributs des portées (request, session) sans code Java.

Expression Language (EL) & JSTL

EL – Syntaxe `${...}`

<!-- Attribut de requête -->

<p>Bienvenue, `${utilisateur.prenom}` !</p>

<!-- Arithmétique -->

<p>Total TTC : `${produit.prixHT * 1.20}` €</p>

<!-- Conditionnel -->

<p>`${utilisateur.age >= 18 ? 'Majeur' : 'Mineur'}`</p>

JSTL – Bibliothèques standard

core (c:)

Conditions, boucles,
variables. La plus utilisée.

fmt (fmt:)

Formatage des dates,
nombres et i18n.

```
<c:forEach var="p" items="${produits}">
```

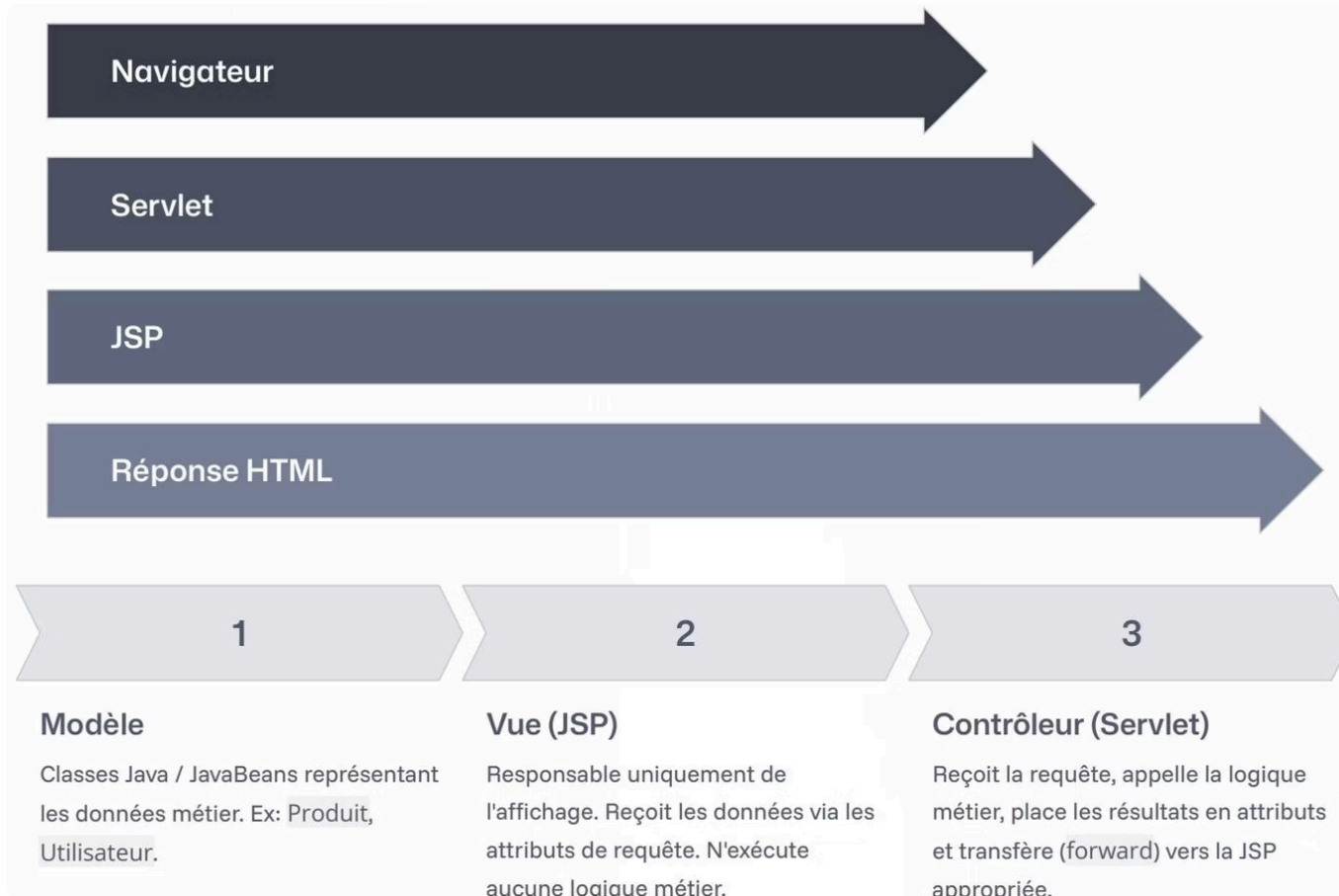
```
<c:if test="${p.stock > 0}">
```

```
<li>${p.nom} — ${p.prix} €</li>
```

```
</c:if>
```

```
</c:forEach>
```

Architecture MVC avec Servlets & JSP



Modèle

Classes Java / JavaBeans représentant les données métier.

Vue (JSP)

Affichage uniquement. Reçoit les données via les attributs de requête.

Contrôleur (Servlet)

Reçoit la requête, appelle la logique métier, transfère vers la JSP.

Forward vs Redirect

Forward (Transfert interne)

L'URL du navigateur **ne change pas**. Les attributs de requête sont préservés. Mécanisme MVC standard.

```
RequestDispatcher rd =  
    request.getRequestDispatcher(  
        "/WEB-INF/vues/liste.jsp");  
rd.forward(request, response);
```

Redirect (Redirection externe)

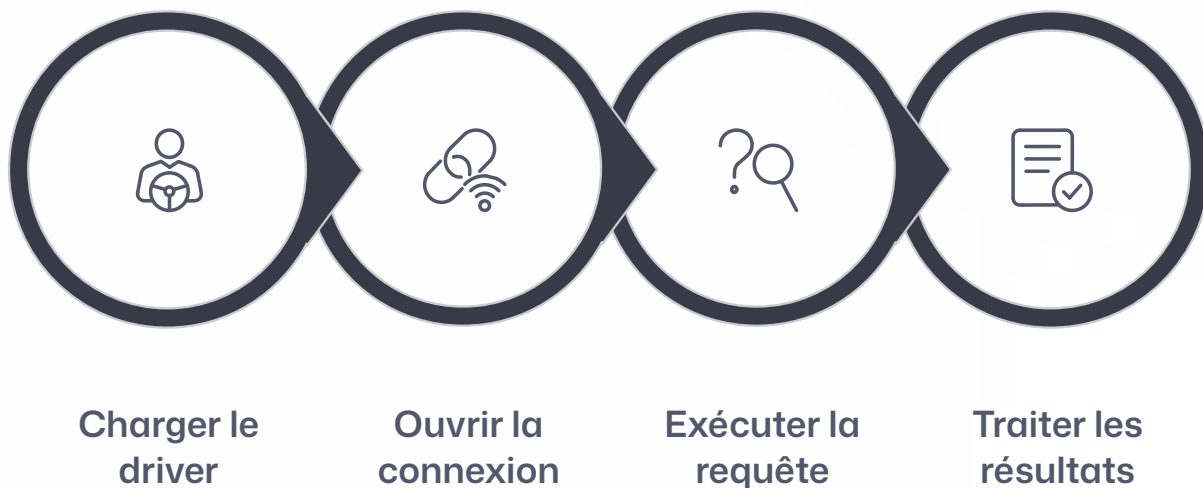
Envoie un HTTP 302 — le navigateur émet une **nouvelle requête**. L'URL change. Utile après un POST (pattern PRG).

```
response.sendRedirect(  
    request.getContextPath()  
    + "/confirmation");
```

- ✓ Placez vos JSP dans `/WEB-INF/vues/` — accessibles uniquement via `forward`, jamais directement.

L'API JDBC

JDBC est l'API standard Java pour interagir avec les bases de données relationnelles (MySQL, PostgreSQL, Oracle...). Le **driver JDBC** spécifique au SGBD implémente les interfaces standard, garantissant la portabilité du code.



PreparedStatement – Requêtes sécurisées

Le **PreparedStatement** précompile la requête SQL et sépare le code des données — protégeant totalement contre l'**injection SQL**.

```
String sql = "SELECT * FROM produits WHERE categorie = ? AND prix < ?";
```

```
try (Connection conn = DriverManager.getConnection(url, user, pwd);
```

```
    PreparedStatement pstmt = conn.prepareStatement(sql)) {
```

```
    pstmt.setString(1, "Électronique");
```

```
    pstmt.setDouble(2, 500.00);
```

```
    ResultSet rs = pstmt.executeQuery();
```

```
    while (rs.next()) {
```

```
        System.out.println(rs.getInt("id") + " - " + rs.getString("nom"));
```

```
    }
```

```
} // Auto-fermeture avec try-with-resources
```

⊗ Ne construisez **jamais** une requête SQL par concaténation avec des données utilisateur. Utilisez toujours `PreparedStatement`.

Pools de connexions

Pourquoi un pool ?

Ouvrir une connexion JDBC est coûteux. Un **pool** maintient des connexions pré-ouvertes : la Servlet en *emprunte* une, l'utilise et la *restitue*.

context.xml

```
<Resource
  name="jdbc/MaBase"
  type="javax.sql.DataSource"
  driverClassName=
    "com.mysql.cj.jdbc.Driver"
  url="jdbc:mysql://localhost/mabase"
  username="root" password="secret"
  maxTotal="20" maxIdle="10"
  minIdle="5"/>
```

Servlet – Lookup JNDI

```
DataSource ds = (DataSource)
  envCtx.lookup("jdbc/MaBase");
try (Connection conn =
  ds.getConnection()) {
  // connexion restituée auto
}
```



Réutilisation

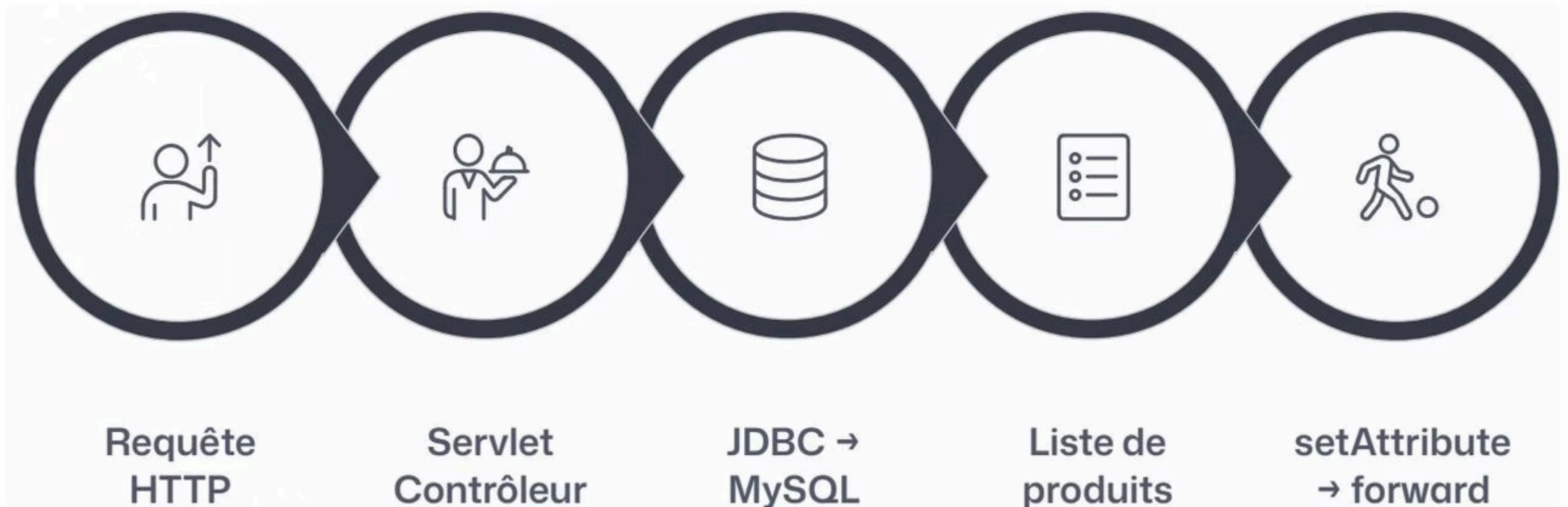
Réduit drastiquement la latence et la charge serveur.



Config Tomcat (JNDI)

Tomcat intègre **DBCP2**. Configurez dans `context.xml`, accédez via JNDI.

Intégration MVC complète – Servlet + JDBC + JSP



```
// Servlet Contrôleur
List<Produit> produits = new ArrayList<>();
String sql = "SELECT id, nom, prix FROM produits ORDER BY nom";

try (Connection conn = ds.getConnection();
    PreparedStatement ps = conn.prepareStatement(sql);
    ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        produits.add(new Produit(rs.getInt("id"),
            rs.getString("nom"), rs.getDouble("prix")));
    }
}
request.setAttribute("produits", produits);
request.getRequestDispatcher("/WEB-INF/vues/produits.jsp")
    .forward(request, response);
```

HTTP/2 & Sécurité

HTTP/2 – Nouveautés

Multiplexage

Plusieurs requêtes simultanées sur une seule connexion TCP.

Compression HPACK

Réduit la surcharge réseau des en-têtes HTTP.

Server Push

Le serveur envoie proactivement CSS/JS avant la demande.



HTTP/2 nécessite **HTTPS** dans la plupart des navigateurs modernes.

Vulnérabilités essentielles

Injection SQL

Protection : toujours `PreparedStatement`.

XSS

Protection : échapper avec `<c:out>` ou `fn:escapeXml()`.

CSRF

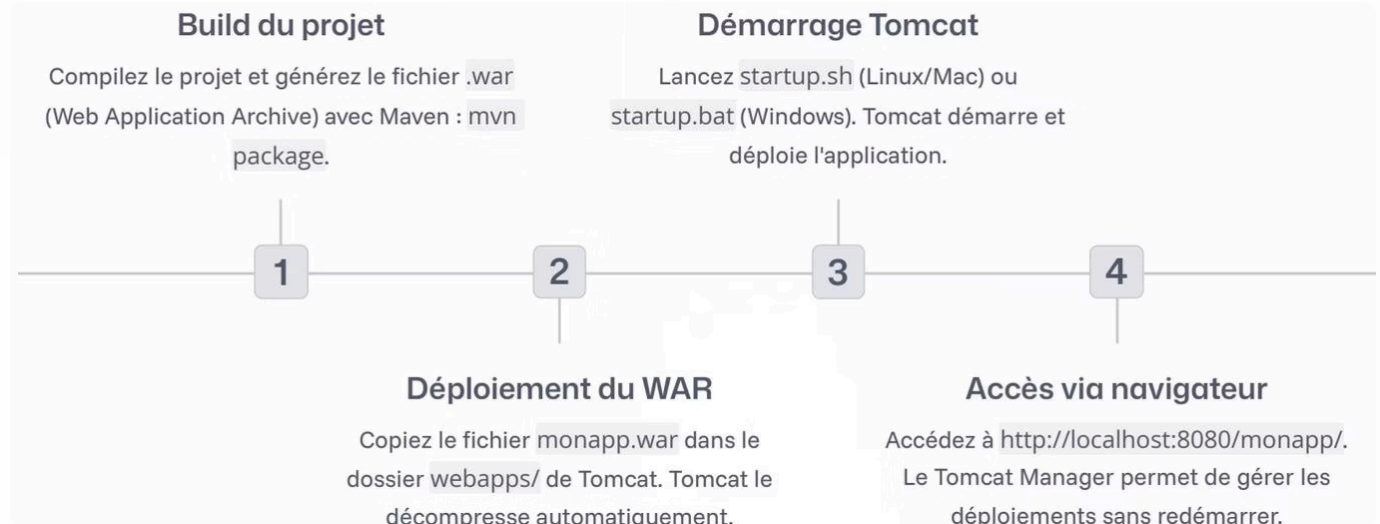
Protection : tokens CSRF dans les formulaires POST sensibles.

Structure du projet & Déploiement

Arborescence standard

```
MonApplication/  
├── src/main/  
│   ├── java/com/monapp/  
│   │   ├── servlets/  
│   │   ├── modeles/  
│   │   └── dao/  
│   └── webapp/  
│       ├── WEB-INF/  
│       │   ├── web.xml  
│       │   └── vues/*.jsp  
│       ├── css/  
│       ├── js/  
│       └── images/  
└── pom.xml
```

Déploiement sur Tomcat



`mvn package` → copier le `.war` dans `webapps/` → Tomcat décompresse automatiquement → accès via `http://localhost:8080/monapp/`.

Le pattern DAO & Gestion des erreurs

Pattern DAO

Centralise tout l'accès JDBC dans des classes dédiées. Définir une **interface** permet de changer l'implémentation sans toucher aux Servlets.

```
public interface ProduitDAO {  
    List<Produit> findAll();  
    Produit findById(int id);  
    void save(Produit p);  
    void delete(int id);  
}
```

Gestion des erreurs

Pages d'erreur (web.xml)

JSP personnalisées pour les codes HTTP 404, 500 — l'utilisateur voit une page soignée.

Try-catch dans les Servlets

Loguez côté serveur, n'exposez jamais les détails techniques à l'utilisateur.

isErrorPage en JSP

`<%@ page isErrorPage="true" %>` donne accès à l'objet implicite `exception`.

Servlet vs JSP – Comparaison

Critère	Servlet	JSP
Rôle MVC	Contrôleur — logique de traitement	Vue — affichage et présentation
Syntaxe	Code Java pur avec annotations	HTML enrichi de JSTL/EL
Idéal pour	Formulaires, redirections, JDBC	Rendu de listes, tableaux HTML
Génération HTML	Via PrintWriter — verbeux	Directement dans le fichier
Compilation	Explicite par le développeur	Automatique par Tomcat

✅ Règle d'or : la **Servlet traite**, la **JSP affiche**. Ne mélangez jamais les responsabilités.

Conclusion & Prochaines étapes

Ce que vous maîtrisez

- Architecture et écosystème Java EE
- Servlets, cycles de vie, sessions, filtres
- JSP, EL, JSTL et pattern MVC
- JDBC, PreparedStatement, pools DBCP2
- HTTP/2, sécurité et bonnes pratiques

Prochaines étapes

→ Application CRUD complète

Pratiquez en construisant un projet de bout en bout.

→ Spring Boot

Industrialisez vos développements avec le framework Java le plus populaire.

→ JPA / Hibernate

Approfondissez la persistance objet-relationnelle.

La maîtrise des fondamentaux Java EE est la clé de voûte de tout développeur web Java. Chaque framework moderne vous semblera familier — car il repose sur ces mêmes briques.